



Event Management System – Deep Analysis Report

Source Document: `Event_Management_System_Documentation.pdf` (7 pages, 8910 chars)

Project Type: Full-Stack Web Application (MERN Stack)

Timeline: 10 Weeks | 7 Phases

Generated: April 26, 2026

1. Executive Summary

The document describes a **full-stack Event Management Platform** that enables users to create, manage, search, and book events with integrated payments and real-time notifications. It follows a **MERN architecture** (MongoDB, Express.js, React.js, Node.js) with a monorepo structure and is designed for three user roles: **Admin**, **Organizer**, and **Attendee**.

The system is production-grade with Stripe payments, Cloudinary media hosting, Socket.io real-time features, QR-code ticketing, and CI/CD deployment to Vercel + Render.

2. Tech Stack Breakdown

Layer	Technology	Purpose
Frontend	React.js (Vite)	SPA with component-based UI
Backend	Node.js + Express.js	REST API server
Database	MongoDB (Mongoose ODM)	Document-based data storage
Auth	JWT (Access + Refresh tokens)	Stateless authentication
Payments	Stripe (Checkout + Webhooks)	Secure payment processing
Real-time	Socket.io	Live ticket updates & notifications
Media	Cloudinary CDN	Image upload, transform, delivery
QR Codes	<code>qrcode</code> npm package	Digital ticket generation

Layer	Technology	Purpose
Email	Nodemailer (Gmail OAuth2/SMTP)	Transactional email delivery
SMS	Twilio (Optional)	Booking/event SMS alerts
Charts	Recharts	Dashboard analytics visualization
Reports	papaparse + pdfkit/puppeteer	CSV/PDF export
Testing	Jest, React Testing Library, MSW, Postman	Unit, component, API tests
CI/CD	GitHub Actions	Automated lint → test → build → deploy
Hosting	Vercel (FE) + Render.com (BE) + MongoDB Atlas	Production deployment

3. Architecture Overview



4. User Roles & Permissions Matrix

Feature	Admin	Organizer	Attendee
Register / Login	✓	✓	✓
Browse & Search Events	✓	✓	✓
Create / Edit / Delete Events	✓	✓ (own only)	✗
Manage Ticket Types	✓	✓ (own events)	✗
Book Tickets & Pay	✓	✓	✓
View Own Bookings	✓	✓	✓
Cancel Bookings	✓	✓	✓
Organizer Dashboard	✓	✓	✗
Admin Dashboard	✓	✗	✗
Manage Users (ban/role change)	✓	✗	✗
Approve Organizer Requests	✓	✗	✗
Export Reports (CSV/PDF)	✓	✓	✗
Receive Real-time Notifications	✓	✓	✓

5. Database Schema Designs

5.1 User Schema

```
{
  name:      { type: String, required: true },
  email:     { type: String, unique: true, required: true },
  password:  { type: String }, // bcrypt hashed, salt rounds: 12
```

```

role:          { type: String, enum: ['admin', 'organizer', 'attendee'],
isVerified:    { type: Boolean, default: false },
refreshToken:  { type: String },
resetOTP:      { type: String },    // hashed OTP
otpExpiry:     { type: Date },
// Suggested additions:
avatar:        { type: String },
phone:         { type: String },
smsOptIn:      { type: Boolean, default: false },
isBanned:      { type: Boolean, default: false },
createdAt:     { type: Date },
updatedAt:     { type: Date }
}

```

5.2 Event Schema

```

{
  title:        { type: String, required: true },    // text indexed
  description:  { type: String },                    // text indexed
  category:     { type: String, enum: ['music', 'tech', 'sports', 'arts',
  date:         { type: Date, required: true },
  location:     { address: String, city: String, coordinates: { lat: Number
  organizer:    { type: ObjectId, ref: 'User' },
  image:        { url: String, publicId: String },
  status:       { type: String, enum: ['draft', 'published', 'cancelled',
  tags:         [{ type: String }],                  // text indexed
  capacity:     { type: Number },
  isDeleted:    { type: Boolean, default: false },    // soft delete
  createdAt:    { type: Date },
  updatedAt:    { type: Date }
}

```

5.3 Ticket Type Schema (per-event)

```

{
  event:        { type: ObjectId, ref: 'Event' },
  name:         { type: String },                    // e.g., 'General', 'VIP', 'Early
  price:        { type: Number },
  quantity:     { type: Number },
  quantitySold: { type: Number, default: 0 },
  saleStart:    { type: Date },

```

```
saleEnd:      { type: Date }
}
```

5.4 Booking Schema

```
{
  user:      { type: ObjectId, ref: 'User' },
  event:     { type: ObjectId, ref: 'Event' },
  tickets:   [{ ticketType: ObjectId, quantity: Number }],
  totalAmount: { type: Number },
  status:    { type: String, enum: ['pending', 'confirmed', 'cancelled'],
  chargeId:  { type: String },           // Stripe charge ID
  qrCode:    { type: String },           // base64 encoded QR
  lockedUntil: { type: Date },           // 15-min payment lock
  createdAt: { type: Date }
}
```

5.5 Notification Schema

```
{
  user:      { type: ObjectId, ref: 'User' },
  type:      { type: String, enum: ['booking_confirmed', 'event_updated'],
  message:   { type: String },
  isRead:    { type: Boolean, default: false },
  relatedId: { type: ObjectId },         // event or booking ID
  createdAt: { type: Date }
}
```

6. Complete API Endpoint Inventory

6.1 Authentication (/api/auth)

Method	Endpoint	Description	Auth
POST	/register	Register new user	Public
POST	/login	Login, returns tokens	Public
POST	/logout	Invalidate refresh token	Protected

Method	Endpoint	Description	Auth
POST	/refresh-token	Rotate access/refresh tokens	Cookie
POST	/forgot-password	Send OTP to email	Public
POST	/reset-password	Verify OTP, update password	Public

6.2 Events (/api/events)

Method	Endpoint	Description	Auth
GET	/	List with search, filter, sort, paginate	Public
GET	/:id	Event detail	Public
POST	/	Create event	Organizer
PUT	/:id	Update event	Organizer (own)
DELETE	/:id	Soft delete event	Organizer (own)
POST	/:id/upload	Upload event image	Organizer (own)

6.3 Bookings & Payments (/api/bookings , /api/payments)

Method	Endpoint	Description	Auth
POST	/bookings/create	Create booking (lock tickets)	Attendee
GET	/bookings/my	User's bookings	Protected
POST	/bookings/:id/cancel	Cancel booking	Protected
POST	/payments/create-intent	Create Stripe PaymentIntent	Protected
POST	/payments/webhook	Stripe webhook handler	Stripe
GET	/tickets/:id/qr	Get QR code for ticket	Protected

Total: 18 documented endpoints (additional CRUD endpoints for admin/dashboard implied)

7. Phase-by-Phase Development Plan

Phase 1: Project Setup (Week 1)

Task	Details	Priority
------	---------	----------

Task	Details	Priority
Initialize monorepo	<code>/client</code> (Vite+React)+ <code>/server</code> (Express)	🔴 Critical
MongoDB connection	Mongoose ODM, connection retry, <code>.env</code> config	🔴 Critical
Environment config	Separate <code>.env</code> files for dev/test/prod	🔴 Critical
Project structure	Backend: routes/controllers/models/middleware/services/utils	🔴 Critical
	Frontend: pages/components/hooks/context/api/assets	🔴 Critical
Linting & formatting	ESLint + Prettier across both projects	🟡 Medium

[!IMPORTANT]

Required env vars: `MONGO_URI` , `JWT_SECRET` , `JWT_EXPIRES_IN` , `CLOUDINARY_*` ,
`STRIPE_SECRET_KEY` , `SMTP_*` , `CLIENT_URL`

Phase 2: Authentication (Week 2)

Task	Details	Priority
Register/Login/Logout	bcrypt (12 salt rounds), email verification	🔴 Critical
JWT implementation	Access: 15min, Refresh: 7d in httpOnly cookie	🔴 Critical
Token rotation	New refresh token on every refresh call	🔴 Critical
Token blacklisting	Store invalidated tokens in Redis/MongoDB	🟡 High
Role middleware	<code>verifyToken</code> → <code>checkRole(roles[])</code> chain	🔴 Critical
Password reset	6-digit OTP, 10min TTL, hashed storage, email delivery	🟡 High

Phase 3: Event Module (Weeks 3–4)

Task	Details	Priority
Event CRUD	Full create/read/update/delete, soft delete	🔴 Critical
Ownership guards	Organizer manages own events only	🔴 Critical
Event statuses	draft → published → cancelled/completed lifecycle	🟡 High
Cloudinary upload	Multer → Cloudinary (1200x630 resize)	🟡 High
Search & filter API	Text index, query params, cursor-based pagination	🔴 Critical

Task	Details	Priority
Event listing UI	Card grid, skeleton loaders, filter sidebar	 Critical
Event detail page	Tabbed sections: info, tickets, map	 High

Phase 4: Ticket & Booking (Weeks 5–6)

Task	Details	Priority
Ticket type management	General/VIP/Early Bird with inventory tracking	 Critical
Booking system	MongoDB transactions, atomic decrement	 Critical
15-min ticket lock	Lock on selection, release on timeout/failure	 High
Stripe Checkout	PaymentIntent → Checkout Session → Webhook	 Critical
Webhook handling	<code>payment_intent.succeeded</code> / <code>failed</code> handlers	 Critical
QR code generation	Unique per ticket, HMAC-signed, base64 in email	 High

[!WARNING]

Critical complexity area: Ticket locking + Stripe webhooks + MongoDB transactions must be bulletproof. Race conditions here = overselling tickets.

Phase 5: Dashboard (Week 7)

Task	Details	Priority
Admin dashboard	KPIs, user management, organizer approval, revenue charts	 High
Organizer dashboard	My events, per-event stats, attendee list, revenue timeline	 High
Analytics charts	Recharts: line (bookings), bar (revenue), pie (ticket types)	 Medium
Report generation	CSV export (papaparse), PDF (pdfkit), email option	 Medium

Phase 6: Real-time Features (Week 8)

Task	Details	Priority
Socket.io setup	Attach to HTTP server, JWT auth on sockets	 High
Room-based updates	Join by event ID, emit on booking changes	 High
Notification system	In-app bell, DB-stored, unread count badge	 High

Task	Details	Priority
Email templates	Booking confirmation, event reminder (24h), OTP reset	🟡 High
SMS alerts (optional)	Twilio, opt-in, feature-flagged, rate-limited	🟢 Low

Phase 7: Testing & Deployment (Weeks 9–10)

Task	Details	Priority
Postman collections	Environment vars, pre-request auth, schema validation	🟡 High
Frontend tests	Jest + RTL, MSW mocking, 70% coverage target	🟡 High
Security hardening	Helmet.js, rate-limit, express-validator, CORS, Morgan	🔴 Critical
CI/CD pipeline	GitHub Actions: lint → test → build → deploy	🟡 High
Production deploy	Vercel (FE) + Render (BE) + MongoDB Atlas + custom domain + SSL	🔴 Critical

8. Frontend Architecture

```
/client
├─ /public
├─ /src
│   ├─ /api                # Axios instance, interceptors, API modules
│   │   ├─ axiosInstance.js
│   │   ├─ authApi.js
│   │   ├─ eventApi.js
│   │   ├─ bookingApi.js
│   │   └─ paymentApi.js
│   ├─ /assets              # Static images, icons, fonts
│   ├─ /components          # Reusable UI components
│   │   ├─ /common          # Button, Input, Modal, Skeleton, Card
│   │   ├─ /layout          # Navbar, Footer, Sidebar, ProtectedRoute
│   │   └─ /events          # EventCard, EventGrid, EventFilters,
EventDetail
│   │   ├─ /booking         # TicketSelector, CheckoutForm, QRDisplay
│   │   ├─ /dashboard       # KPICard, ChartContainer, DataTable
│   │   └─ /notifications   # NotificationBell, NotificationList
│   └─ /context              # React Context providers
```

```
| | └─ AuthContext.jsx
| | └─ SocketContext.jsx
| | └─ NotificationContext.jsx
| └─ /hooks                # Custom hooks
| | └─ useAuth.js
| | └─ useEvents.js
| | └─ useSocket.js
| | └─ useDebounce.js
| └─ /pages                # Route-level page components
| | └─ Home.jsx
| | └─ Login.jsx / Register.jsx
| | └─ Events.jsx / EventDetail.jsx
| | └─ CreateEvent.jsx
| | └─ Checkout.jsx
| | └─ MyBookings.jsx
| | └─ AdminDashboard.jsx
| | └─ OrganizerDashboard.jsx
| | └─ Profile.jsx
| └─ /utils                # Helper functions
| └─ App.jsx
└─ main.jsx
└─ .env
└─ vite.config.js
└─ package.json
```

Key Frontend Pages (10 core pages)

Page	Route	Access
Home / Landing	/	Public
Login	/login	Public
Register	/register	Public
Event Listing	/events	Public
Event Detail	/events/:id	Public
Create/Edit Event	/events/new , /events/:id/edit	Organizer
Checkout	/checkout/:eventId	Authenticated
My Bookings	/bookings	Authenticated
Admin Dashboard	/admin	Admin

Page	Route	Access
Organizer Dashboard	/dashboard	Organizer

9. Backend Architecture

/server

```

├─ /config
│   ├── db.js           # MongoDB connection with retry
│   ├── cloudinary.js    # Cloudinary config
│   ├── stripe.js        # Stripe instance
│   └─ socket.js         # Socket.io setup
├─ /controllers
│   ├── authController.js
│   ├── eventController.js
│   ├── bookingController.js
│   ├── paymentController.js
│   ├── dashboardController.js
│   └─ notificationController.js
├─ /middleware
│   ├── auth.js          # verifyToken, checkRole
│   ├── upload.js        # Multer config
│   ├── errorHandler.js  # Global error middleware
│   ├── rateLimiter.js   # express-rate-limit
│   └─ validator.js      # express-validator rules
├─ /models
│   ├── User.js
│   ├── Event.js
│   ├── TicketType.js
│   ├── Booking.js
│   └─ Notification.js
├─ /routes
│   ├── authRoutes.js
│   ├── eventRoutes.js
│   ├── bookingRoutes.js
│   ├── paymentRoutes.js
│   └─ dashboardRoutes.js
├─ /services
│   ├── emailService.js  # Nodemailer templates
│   └─ smsService.js     # Twilio (optional)

```

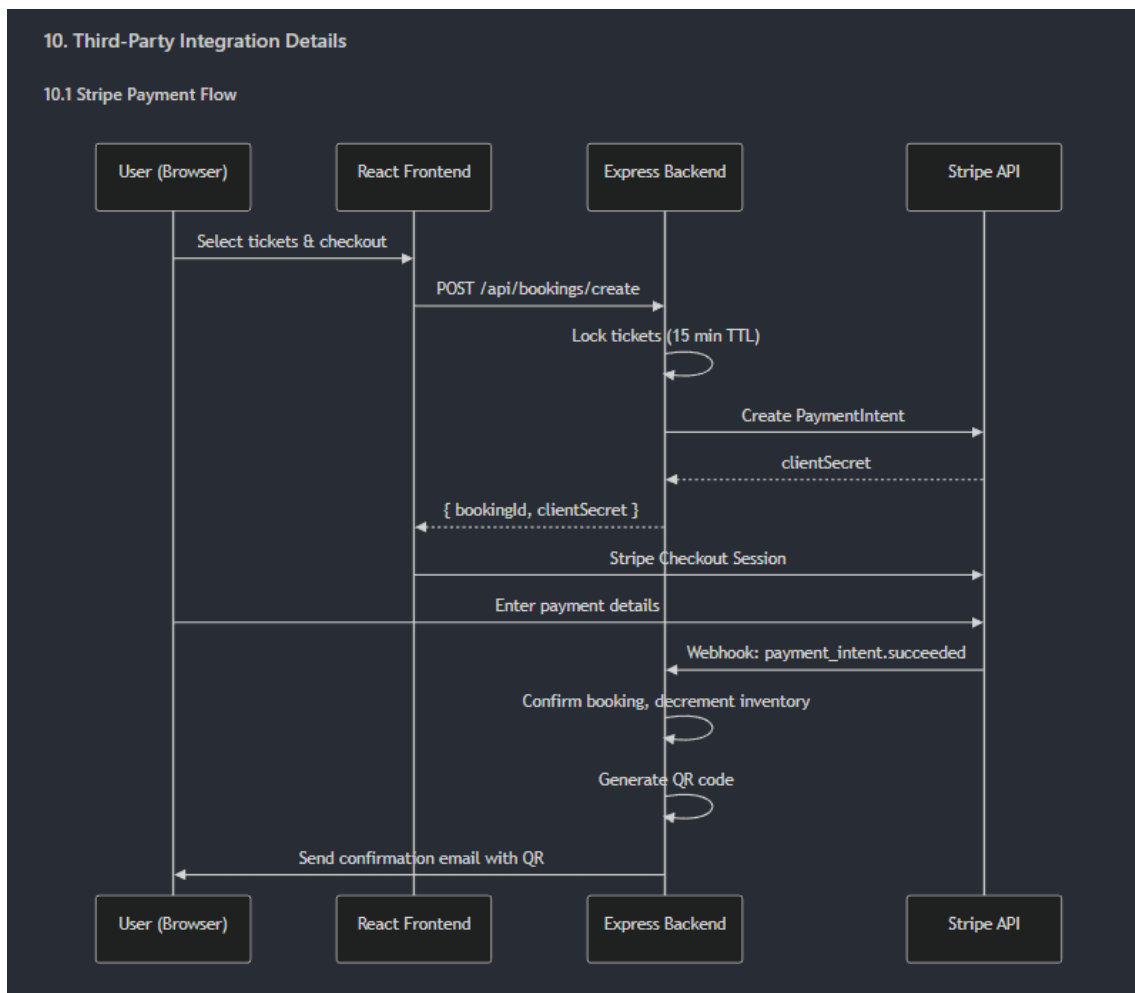
```

|   ├── qrService.js           # QR code generation
|   └── reportService.js       # CSV/PDF generation
└── /utils
    ├── AppError.js           # Custom error class
    ├── asyncHandler.js       # try-catch wrapper
    └── helpers.js
server.js                      # Entry point
.env
package.json

```

10. Third-Party Integration Details

10.1 Stripe Payment Flow



10.2 Cloudinary Image Pipeline

- **Upload:** Multer parses multipart → stream to Cloudinary
- **Transform:** Auto-resize to 1200×630 on upload
- **Store:** Save `secure_url` + `public_id` in Event document
- **Cleanup:** Delete old image from Cloudinary on event image update

10.3 Socket.io Real-time Architecture

- Socket server attached to same HTTP server
- JWT authentication middleware on socket handshake
- Room-based: each event = one room (`event:{eventId}`)
- Events emitted: `ticket:updated` , `booking:confirmed` , `notification:new`

11. Security Checklist

Area	Implementation	Status
Password hashing	bcrypt, 12 salt rounds	Required
JWT tokens	Short-lived access (15min)+ httpOnly refresh (7d)	Required
Token rotation	New refresh token on every refresh call	Required
Token blacklisting	Redis/MongoDB invalidated token store	Required
HTTP headers	Helmet.js	Required
Rate limiting	express-rate-limit on auth & payment routes	Required
Input validation	express-validator on all inputs	Required
CORS	Whitelist production domain only	Required
Webhook verification	Stripe signature verification	Required
QR security	HMAC signature on bookingId + ticketId	Required
SQL/NoSQL injection	Mongoose schema validation + sanitization	Required
File upload	Multer file type/size restrictions	Required
Session invalidation	On password reset, invalidate all sessions	Required

12. Risk Areas & Mitigation

Risk	Impact	Mitigation
Ticket overselling	 Critical – financial/trust	MongoDB transactions + atomic operations + 15-min lock
Stripe webhook failures	 Critical – orphaned bookings	Idempotent webhook handler + retry logic + manual reconciliation

Risk	Impact	Mitigation
JWT token leakage	🔴 High – account compromise	Short-lived access tokens, httpOnly cookies, token rotation
Cloudinary costs	🟡 Medium – budget overrun	Image size limits, transformation presets, CDN caching
Socket.io scaling	🟡 Medium – connection limits	Redis adapter for multi-instance, connection pooling
Email deliverability	🟡 Medium – OTPs not received	Gmail OAuth2, fallback SMTP, SPF/DKIM records

13. NPM Dependencies Summary

Backend (/server/package.json)

express, mongoose, jsonwebtoken, bcryptjs, dotenv, cors, helmet, morgan, express-rate-limit, express-validator, multer, cloudinary, stripe, socket.io, nodemailer, qrcode, papaparse, pdfkit, twilio (optional)

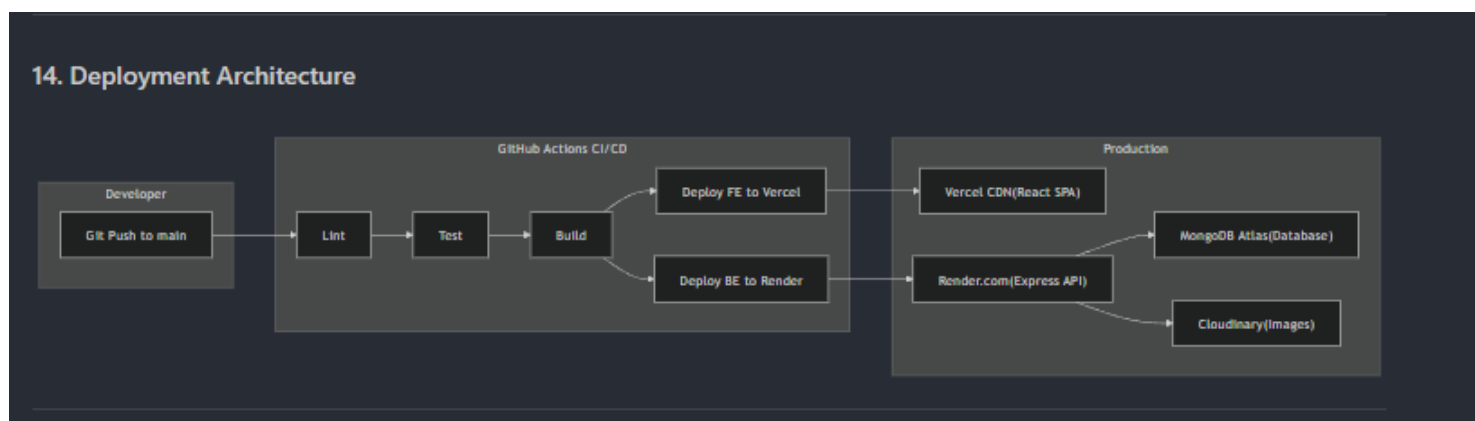
Frontend (/client/package.json)

react, react-dom, react-router-dom, axios, socket.io-client, @stripe/stripe-js, @stripe/react-stripe-js, recharts, react-icons, react-toastify, date-fns or dayjs, react-datepicker

Dev Dependencies

eslint, prettier, jest, @testing-library/react, @testing-library/jest-dom, msw, vite

14. Deployment Architecture



15. Recommended Implementation Order

[!TIP]

Follow this exact order to minimize blockers and ensure each phase builds on a stable foundation.

- Week 1 → Project scaffolding, MongoDB connection, env config, folder structure
- Week 2 → Auth system (register/login/JWT/roles/password reset)
- Week 3 → Event model + CRUD API + Cloudinary integration
- Week 4 → Event listing UI + search/filter/pagination + detail page
- Week 5 → Ticket types + booking system + MongoDB transactions
- Week 6 → Stripe integration + webhooks + QR code generation
- Week 7 → Admin & Organizer dashboards + Recharts + CSV/PDF export
- Week 8 → Socket.io real-time + notifications + email templates
- Week 9 → Testing (Postman + Jest + RTL) + security hardening
- Week 10 → CI/CD pipeline + deploy to Vercel/Render + custom domain + SSL

16. Open Questions & Decisions Needed

#	Question	Options
1	State management?	React Context (documented) vs. Redux Toolkit vs. Zustand
2	CSS framework?	Tailwind CSS vs. CSS Modules vs. Styled Components
3	Token blacklisting store?	Redis (better perf) vs. MongoDB collection (simpler)
4	Pagination approach?	Cursor-based (documented) vs. offset-based (simpler)
5	Email provider?	Gmail OAuth2 (dev) vs. SendGrid/Mailgun (production)
6	Map integration?	Google Maps vs. Mapbox vs. Leaflet (for event location)
7	SMS alerts scope?	Include Twilio in MVP or defer to post-launch?
8	Admin approval flow?	Auto-approve organizers or require manual admin approval?

[!NOTE]

This report is a living document. As development progresses, each phase section should be updated with actual implementation decisions, blockers encountered, and deviations from the original plan.